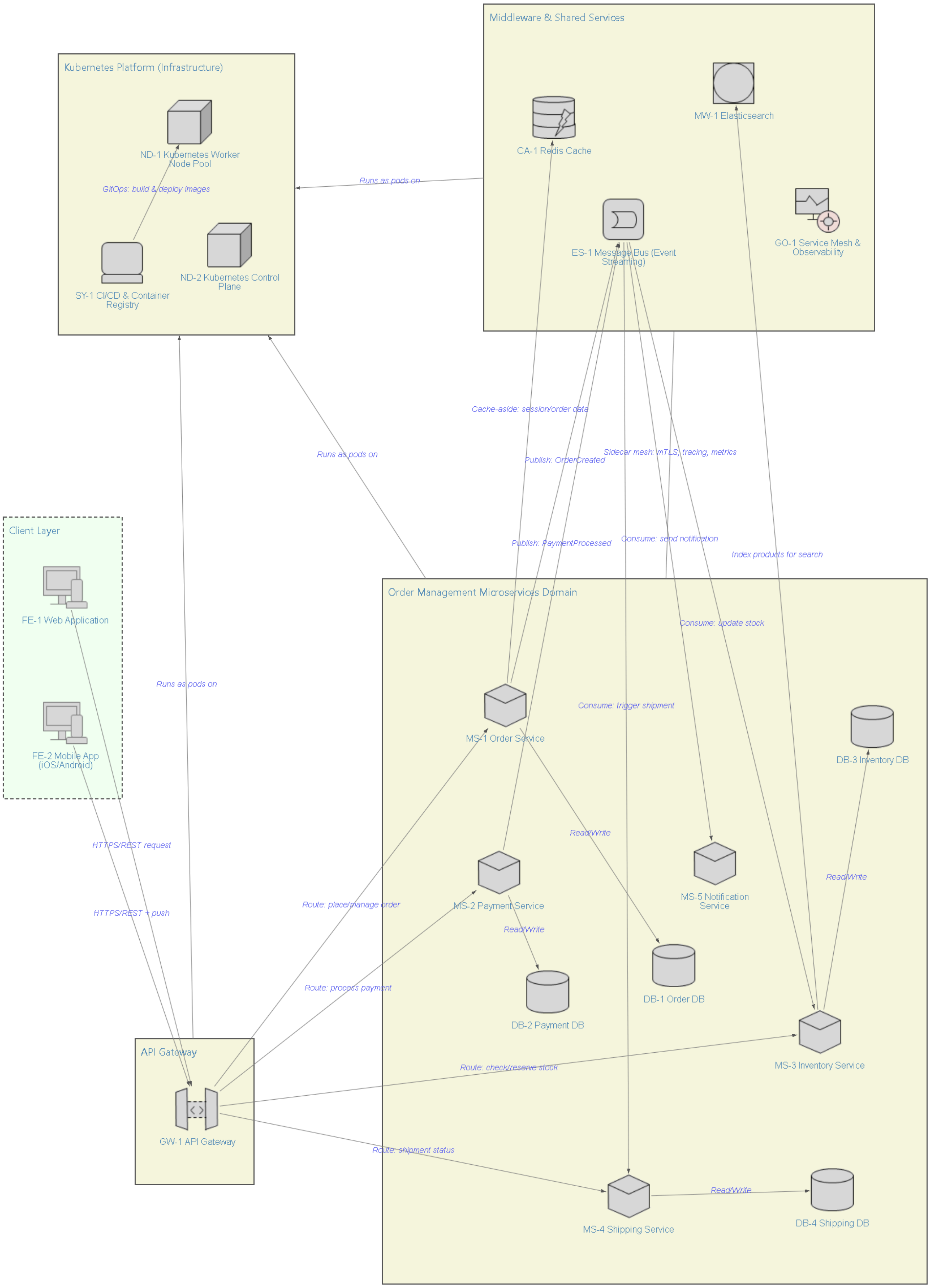
Middleware & Shared Services
Kubernetes Platform (Infrastructure)
ND-1 Kubernetes Worker Node Pool
GitOps: build & deploy images
SY-1 CI/CD & Container Registry
ND-2 Kubernetes Control Plane
Runs as pods on
CA-1 Redis Cache
MW-1 Elasticsearch
ES-1 Message Bus (Event Streaming)
GO-1 Service Mesh & Observability
Cache-aside: session/order data
Sidecar mesh: mTLS, tracing, metrics
Publish: OrderCreated
Runs as pods on
Publish: PaymentProcessed
Consume: send notification
Index products for search
Client Layer
FE-1 Web Application
FE-2 Mobile App (iOS/Android)
Order Management Microservices Domain
Consume: update stock
Runs as pods on
MS-1 Order Service
Consume: trigger shipment
DB-3 Inventory DB
Read/Write
HTTPS/REST request
MS-5 Notification Service
MS-2 Payment Service
Read/Write
HTTPS/REST + push
Route: place/manage order
Read/Write
DB-1 Order DB
DB-2 Payment DB
Route: process payment
API Gateway
Route: check/reserve stock
MS-3 Inventory Service
GW-1 API Gateway
Route: shipment status
Read/Write
MS-4 Shipping Service
DB-4 Shipping DB

View Properties

Property	Attribute
Title	Microservice Architecture Pattern View - Order Management (DevOps Perspective)
Area	Overall-solution View
Description	This is an overall-solution pattern view of a typical microservice architecture, illustrated through an Order Management domain, from a DevOps/operational lens. **Key requirements and quality attributes:** 1) elastic scalability per service under variable order volume, 2) resilience/availability via service isolation and async messaging so a single service failure does not cascade, 3) observability (tracing, metrics, centralized search/logs) for fast root-cause diagnosis. **Architectural techniques:** API Gateway pattern (single entry point, AuthN/AuthZ, rate limiting), database-per-service (polyglot persistence), cache-aside with Redis, event-driven integration via a message bus (Kafka-style pub/sub) to decouple services, and container-orchestrated deployment on Kubernetes with a service mesh for traffic control, mTLS, and telemetry. **Key trade-offs:** event-driven decoupling improves resilience and independent scaling but adds eventual-consistency and debugging complexity; the API Gateway simplifies client access but is a critical path requiring HA and its own scaling.
Status	Draft - Pattern Reference View, for architect/DevOps review
Implication	This view assumes a standard e-commerce Order Management context and is meant as a reusable pattern, not a specific vendor design. Related views to complete the model: an **Operational view** detailing network zones, security groups, and multi-region/HA topology; a **Functional view** detailing each microservice internal API contracts and data schema; and a **Metrics view** defining concrete SLOs (latency, throughput, error budget) per service. Specific product choices (Kong vs NGINX, Kafka vs RabbitMQ, EKS vs GKE) are illustrative and should be confirmed against organizational standards.
Timestamp	09/03/26, 12:16:26

Element Properties

Element	Property	Attribute
FE-1 Web Application	Type	SPA (React)
	Protocol	HTTPS/REST
	Auth	OAuth2/JWT (via Gateway)
	Users	Customers, Store Admins
FE-2 Mobile App (iOS/Android)	Type	Native app
	Protocol	HTTPS/REST, Push
	Auth	OAuth2/JWT (via Gateway)
	Offline	Local cache for cart/session
GW-1 API Gateway	Protocol	HTTPS/REST, gRPC
	Responsibilities	AuthN/AuthZ, rate limiting, routing, TLS termination
	Tech	Kong/NGINX/Envoy
	SLA target	99.9% availability
MS-1 Order Service	Runtime	Java/Spring Boot
	Pattern	Domain-driven design, orchestrator of order lifecycle
	Scaling	Horizontal (HPA on CPU/queue depth)
	API	REST + async events
DB-1 Order DB	Engine	PostgreSQL
	Topology	Primary + read replicas

Element	Property	Attribute
	Backup	Daily snapshot, PITR
MS-2 Payment Service	Runtime	Go
	Pattern	Saga (distributed transaction)
	Compliance	PCI-DSS scope, isolated network policy
	API	REST
DB-2 Payment DB	Engine	PostgreSQL
	Topology	Primary + read replicas
	Encryption	At rest and in transit
MS-3 Inventory Service	Runtime	Node.js
	Consistency	Eventual (event-updated stock counts)
	Scaling	Horizontal (HPA)
	API	REST
DB-3 Inventory DB	Engine	MongoDB
	Topology	Sharded cluster
	Model	Document/NoSQL
MS-4 Shipping Service	Runtime	Python
	Integration	Third-party carrier APIs
	API	REST
DB-4 Shipping DB	Engine	PostgreSQL
	Topology	Single primary + replica
MS-5 Notification Service	Runtime	Node.js
	Channels	Email, SMS, push
	Trigger	Event-driven (async only, no direct sync calls)
CA-1 Redis Cache	Purpose	Session cache, rate-limit counters, hot product/order data
	Mode	Clustered
	Eviction	LRU
MW-1 Elasticsearch	Purpose	Product search index, centralized log aggregation
	Mode	Clustered (3+ nodes)
	Indices	Products, application logs
ES-1 Message Bus (Event Streaming)	Tech	Kafka-style pub/sub
	Pattern	Event-driven integration, decoupled services
	Key topics	OrderCreated, PaymentProcessed, ShipmentDispatched
GO-1 Service Mesh & Observability	Tech	Istio/Envoy sidecars, Prometheus, Grafana, Jaeger
	Function	mTLS, traffic mgmt, distributed tracing, metrics
ND-1 Kubernetes Worker Node Pool	OS	Linux (containerd)
	Scaling	Cluster Autoscaler
	Compute	Multi-AZ VM instances
	Workload	Microservice + gateway pods
ND-2 Kubernetes Control Plane	Managed	EKS/GKE/AKS style
	HA	Multi-master, multi-AZ
	Function	API server, scheduler, etcd
SY-1 CI/CD & Container Registry	Tools	Docker, Helm, ArgoCD/Jenkins

Element	Property	Attribute
	Function	Build, image registry, GitOps deployment to cluster

Element Prefix Notation

Prefix	Element Name	Relevance	Category
CA	Cache Service	Assistive	Integration & Scalability
DB	Data Store	Base	Operational / Infrastructure
ES	Event Service	Assistive	Architectural Style
FE	Frontend	Assistive	Architectural Style
GO	Governance Control	Assistive	DevOps
GW	Gateway Service	Assistive	Integration & Scalability
MS	Microservice	Assistive	Architectural Style
MW	Middleware	Base	Operational / Infrastructure
ND	Node	Base	Operational / Infrastructure
SY	System	Base	Operational / Infrastructure